Nainisha Bandari
Vagdevi Malineni

OAUTH (GOOGLE LOGIN) AND SESSION HANDLING

1. OAuth (Google Login)

Definition:

OAuth (Open Authorization) is an authentication and authorization framework that allows users to log in using their Google account without sharing their password directly with the application.

Purpose of Google OAuth Login:

- Simplifies user authentication
- Improves security
- Reduces password management complexity
- Provides faster signup/login experience
- Allows access to verified Google user information

Components Required:

Component	Purpose
Google Cloud Console	Create OAuth credentials
Client ID	Identifies the application
Client Secret	Verifies the application securely
Redirect URI	URL where Google sends the user after login
OAuth Consent Screen	Displays app details to users

Google OAuth Login Flow:

1. User clicks “Login with Google”
2. Application redirects user to Google Authentication Page
3. User enters Google credentials
4. Google verifies the user
5. Google asks for permission consent
6. Google sends authorization code/token to application
7. Application validates token
8. User session is created
9. User gains access to the application

Basic OAuth Setup Process:

Step 1: Create Google Cloud Project

- Open Google Cloud Console
- Create a new project

Step 2: Enable OAuth API

- Enable Google Identity Services or OAuth API

Step 3: Configure OAuth Consent Screen

Provide:

- Application name
- Support email
- Authorized domains
- Privacy policy URL

Step 4: Generate Credentials

Create:

- OAuth Client ID
- Client Secret

Step 5: Add Redirect URI

http://localhost:3000/auth/google/callback

Example Backend Setup (Node.js + Express)

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

passport.use(new GoogleStrategy({
    clientID: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL: '/auth/google/callback'
}, (accessToken, refreshToken, profile, done) => {
    return done(null, profile);
}));
```

2. Session Handling

Definition

- Session handling is the process of storing and managing user login state after authentication.
- A session allows the server to remember the logged-in user across multiple requests.

Purpose of Session Handling

- Keeps users logged in
- Maintains authentication state
- Prevents repeated login requests
- Secures user access
- Tracks user activity

Session Workflow

1. User logs in successfully
2. Server creates a session
3. Session ID is generated
4. Session ID is stored in browser cookie
5. Browser sends session ID with every request
6. Server validates session
7. User remains authenticated until logout or expiration

Types of Session Management

Type	Description
Server-side Session	Session stored on server
Cookie-based Session	Session stored in browser cookies
JWT Token Session	Authentication managed using tokens

Session Lifecycle:

Step 1: Login Success

User authenticates successfully.

Step 2: Session Created

Server generates:

- Session object
- Session ID

Step 3: Cookie Generated

Session ID stored in browser cookie.

Example:

connect.sid=abc123xyz

Step 4: Browser Sends Cookie

Cookie sent with every request.

Step 5: Server Validation

Server checks session store.

Step 6: User Authorized

If session exists, access is granted.

Step 7: Session Expiry or Logout

Session destroyed after:

- Logout
- Timeout
- Manual invalidation

Session Setup Using Express Session

Install Dependency

npm install express-session

Example Setup:

```
const session = require('express-session');

app.use(session({
    secret: 'mySecretKey',
    resave: false,
    saveUninitialized: false,
    cookie: {
        secure: false,
        maxAge: 60000
    }
}));
```

OAuth + Session Integration Workflow

Combined Authentication Process

OAuth Phase

1. User authenticates with Google
2. Google returns user information

Session Phase

3. Server creates session
4. Session ID stored in cookie
5. Browser automatically sends session cookie
6. User remains authenticated